Bengaluru Flood-Risk Map using MPI

Running 400 rainstorms over a real map of Bengaluru to find the places that flood most often, with the work shared across many processes

Course	Parallel Computing (PCAP) - Project Synopsis
Programme	B.Tech, Computer Science and Engineering
Made by	Atharv Remeshan - 245805386 Pranshu Thakkar - 245805158 Ritwik Gupta - 245805240
Tools	C with OpenMPI; Python (NumPy, Matplotlib) for the maps and charts; raylib for the live demo

1. Summary

Bengaluru floods almost every monsoon. Roads like the Outer Ring Road and junctions like Silk Board turn into small lakes. Rain water always runs downhill and collects in low places, so the shape of the land already tells us a lot about where water will go.

We took a real height map of Bengaluru from satellite data and let 400 different rainstorms fall on it, from short light showers to long heavy downpours. A simple computer model moves the rain water downhill, step by step. For every small patch of land, we record how deep the water gets. The result is a colour-coded flood map of the city, a list of the most flood-prone areas, and a check against the places the city corporation (BBMP) has officially listed as flood-prone.

The goal of the project is to learn parallel programming. The 400 storms do not depend on each other, so many processes can work on different storms at the same time. We wrote all the code that shares out the work and collects the results ourselves, using MPI. The map shows where flooding is likely; it is not a weather forecast.

2. What the program does

Step	In plain words
1. Load the map	Read the ground height of every patch of land. The map has 1.33 million patches, each 30 m x 30 m, covering about 1,200 km^2 of Bengaluru. We use FABDEM satellite data (buildings and trees removed) and also cut about 990 km of known drains into it
2. Make the storms	Create 400 storms: 20 rain strengths (10 to 60 mm per hour) times 20 lengths (1 to 4 hours)
3. Run each storm	Rain falls everywhere. Every 30 seconds of storm time, water moves from each wet patch to its lower neighbours. After the rain stops, we let the water drain for 2 more hours
4. Combine the storms	For every patch, count how often it flooded and remember the deepest water it ever had
5. Show the results	Draw the flood map, rank 500 m areas by risk, compare with 386 official BBMP flood spots, and make a table of flooded area per storm

Table 1: The five steps of the program.

3. Why we use parallel processing

- **There is a lot of work.** 400 storms, each with up to 720 time steps, on 1.33 million patches: billions of small calculations. One storm takes about 4 to 14 seconds on one processor core, so all 400 take about an hour on one process.
- **The work splits easily.** Each storm is completely separate from the others. Every process gets its own copy of the map and runs whole storms by itself. The processes only need to talk at the start (to get the settings) and at the end (to add up the results).
- **But storms are not the same size.** A 4-hour storm takes about twice as long as a 1-hour storm. If we simply give each process an equal number of storms, some finish early and sit idle while others are still busy. Sharing the work fairly is the interesting problem in this project.
- **Some parts cannot be sped up.** Adding up the results at the end, loading the map and drawing the pictures take the same time however many processes we use.

4. How the processes work together

4.1 Sharing the settings

Process 0 reads the settings (which map, which storms) and sends them to all other processes. Each process then loads the map from disk by itself, so the big map is never sent between processes. Only storm numbers are sent.

4.2 Handing out storms one at a time (dynamic mode)

Process 0 acts as a manager and does not run storms itself. Each worker tells the manager "I am ready" (and sends the result of its last storm), and the manager replies with the next storm number, or "stop" when none are left. A worker that gets a short storm simply comes back sooner, so nobody waits.

4.3 Fixed shares (static mode, for comparison)

Each worker gets a fixed list of storms at the start and never talks to the manager. This is simpler, but it cannot adjust when some storms take longer than others.

4.4 Collecting the results

At the end, the flood counts from all workers are added together, patch by patch, and the deepest water is kept. We also collect how long each worker was busy, so we can see how evenly the work was shared.

4.5 Checking the answer

The model has no randomness, so the parallel version must give exactly the same answer as the one-process version. We compare the output files byte by byte.

4.6 Live demo

A separate window shows one storm happening on a street map of Bengaluru. You can watch the water spread, zoom in, and point at any spot to see its height and water depth.

5. MPI features used

MPI feature	What we use it for
Broadcast	Process 0 sends the same settings to every process
Send and receive	Workers ask for the next storm and the manager answers whoever asks first
Reduce	Adds up the flood counts from all processes (sum) and keeps the deepest water (maximum)
Gather	Collects how long each process was busy
Barrier	Makes all processes start and stop the clock together
Timer	Measures how long the run takes

Table 2: The MPI features we use.

6. What we found

6.1 The parallel answer is exactly right

With 2 and 4 processes, in both modes, the output files are exactly the same as with one process, down to the last byte. The model also loses almost no water (less than 0.0003% error).

6.2 Speed

We timed 400 storms on a smaller 90 m version of the map, three times each, and used the middle value. "Speed-up" means how many times faster than one process. "Imbalance" means how much more work the busiest worker did than the average worker (1.00 = perfectly fair).

Processes	Dynamic	Static	Speed-up (dynamic / static)	Imbalance (dynamic / static)
1	402 s	-	1.0x	-
4	246 s	253 s	1.6x / 1.6x	1.00 / 1.04
8	170 s	178 s	2.4x / 2.3x	1.01 / 1.05
14	131 s	138 s	**3.1x / 2.9x**	**1.02 / 1.10**

Table 3: Speed-up and imbalance.

- **Dynamic mode shares the work more fairly.** Its busiest worker does only 2% more than average; with fixed shares it is 8 to 10%. Dynamic was also a little faster (about 5%), but a second test showed that timings on this laptop change by 10 to 20% from run to run, so the fairness numbers are the clearer result.
- **Why only about 3 times faster with 14 processes?** Because of the laptop, not MPI. One storm takes 1.0 s when it runs alone, but 4.1 s when 13 storms run at once. All processes share the same memory, cache and power, and some of the CPU cores are slower "efficiency" cores. The MPI messages themselves are tiny.
- The full 400-storm run on the detailed 30 m map takes about 14 minutes with 12 processes, instead of about an hour with one.

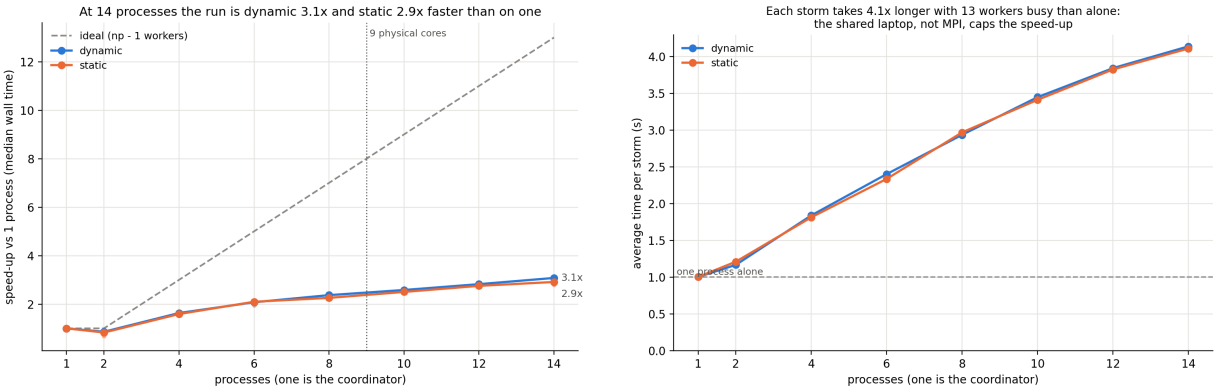


Figure 1: Left, how much faster the program gets (the dashed line is the perfect case). Right, each storm gets slower as more processes share the laptop.

6.3 Where Bengaluru floods

- Flooding follows the city's natural low lines: valleys, drains and chains of lakes. Hot spots are in the west and centre (Vijayanagar, Malleshwaram, Rajajinagar) and around the south-east lakes (Bellandur, Outer Ring Road).
- Stronger rain floods much more land: 13.5 km^2 for the lightest storm (10 mm/h for 1 hour), 58 km^2 for 60 mm/h for 1 hour, and 191 km^2 for the heaviest (60 mm/h for 4 hours). How hard it rains matters more than how long.
- **87 of the 384 official BBMP flood spots fall in our riskiest 10% of areas. That is 2.3 times more than random places would.** 6 of our 10 riskiest areas have a BBMP flood spot within 1 km.

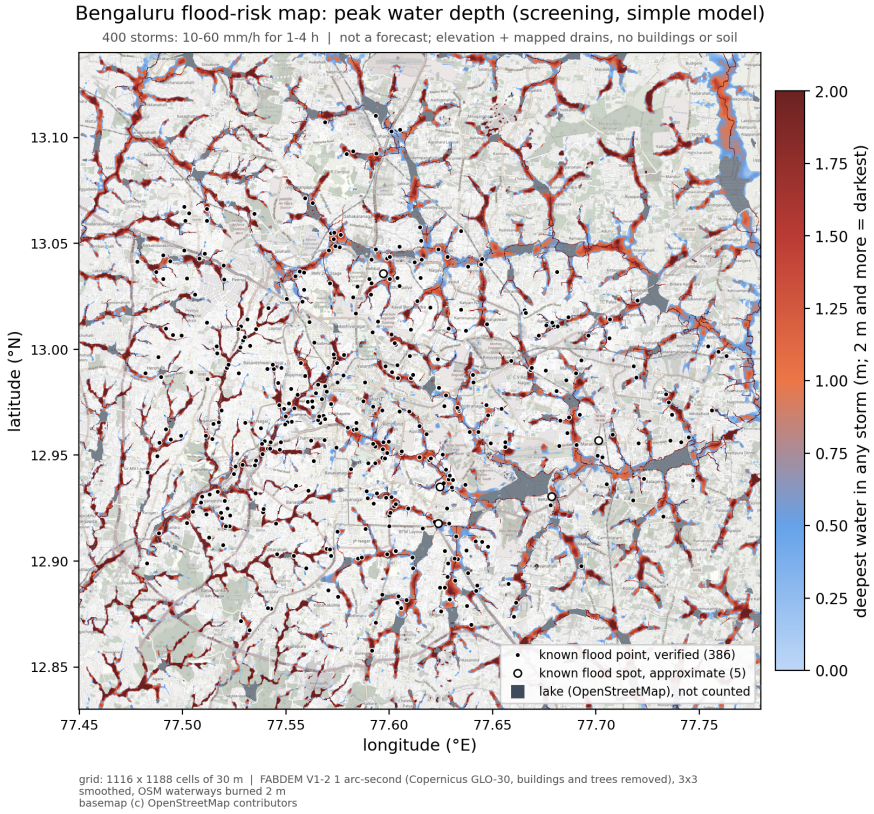


Figure 2: The deepest water each place reached in any of the 400 storms, on a street map. Black dots: BBMP flood spots. Grey: lakes.

6.4 Making the map more accurate

We tested each improvement fairly: we chose settings using half of the BBMP spots and measured the result on the other half. The score below is the chance that a real flood spot is rated riskier than a random place (0.5 = guessing, 1.0 = perfect).

Version of the model	Score on the unseen half
Original height map (SRTM, includes rooftops and trees)	0.55
Better height map (FABDEM, buildings and trees removed)	0.68
+ about 990 km of known drains added	0.71
+ judging risk by the deepest water during the storm	0.71

Table 4: Each improvement and its score.

We also tried letting rain soak into the ground and using a wider range of storms. Neither made the map more accurate.

6.5 Limits

This map shows where flooding is likely; it is not a forecast. The model does not know about blocked or small drains, soil, buildings or underpasses. So about 7 out of 10 real flood spots still fall outside our riskiest areas, and the water depths are model numbers, not exact predictions.

7. Tools

Language	C, built with the MPI compiler (mpicc)
MPI library	OpenMPI 4.1
Height map	FABDEM (30 m, buildings and trees removed, University of Bristol); SRTM for comparison
Flood spots	386 official BBMP flood locations (open data from OpenCity)
Map data	OpenStreetMap: drains, lakes and the street map, (c) OpenStreetMap contributors
Live demo	raylib, a small C graphics library
Computer	14-core laptop (Intel Core Ultra 5 125H), Ubuntu 24.04 under WSL; tested with up to 14 processes
Charts	Python with NumPy and Matplotlib